

Universal 12-Week Software Engineering Roadmap

A Student-Friendly, Step-by-Step Implementation Guide for Capstone & Final Year Projects

STUDENT SUCCESS PROTOCOL & ENCOURAGEMENT

- Welcome to your capstone software engineering development lab. Building a complete software product across 12 weeks is structured into simple, manageable weekly steps.
- You are encouraged to design your own creative features and custom workflows. If you ever feel stuck, use the premade starter blueprints in this document to stay unblocked.
- Focus on completing High Priority deliverables to guarantee you pass. Completing Medium Priority deliverables establishes a strong engineering foundation for honors and career interviews.

Table of Contents

Click any section below to jump directly to its roadmap and instructions:

Section 1: The 3-Iteration Development Framework — Overview of the 3-iteration lifecycle, roadmap timeline, and prototype gate.

Section 2: Iteration 1 - Foundation & Working Prototype (Weeks 01 - 07) — Requirements, system architecture, database modeling, auth, and the Week 07 Prototype Gate.

- [Week 01: Problem Discovery & User Personas](#) — Stakeholder interviews, user personas, problem statement, and scope definition.

- [Week 02: Software Requirements Specification \(SRS\)](#) — Formal IEEE 830 SRS document, use cases, and requirements traceability.

- [Week 03: High-Level Architecture & Cloud Topology](#) — 3-Tier architecture diagrams, Data Flow Diagrams (DFD Level 0/1), and API boundaries.

- [Week 04: Database Modeling & UI Wireframes](#) — Database ERD / Mongoose models, API endpoint contracts, and screen wireframes.

- [Week 05: Project Scaffolding & Core Auth Layer](#) — Repository setup, folder structure, database connection, and JWT user authentication.

- [Week 06: Core Business Logic & Vertical Slice](#) — Primary calculation engine, input validation, and end-to-end frontend-to-database connection.

- [Week 07: Live Working Prototype Milestone Gate](#) — Mid-term working prototype demonstration, smoke testing, and supervisor progress review.

Section 3: Iteration 2 - Feature Expansion & Governance (Weeks 08 - 09) — Secondary feature modules, third-party APIs, admin portal, and consumer UX polish.

- [Week 08: Secondary Modules & Third-Party APIs](#) — Gemini AI, WhatsApp, Stripe, Maps, background tasks, and resilient error fallbacks.

- [Week 09: Admin Workstation & Consumer UX Polish](#) — Protected Admin portal with RBAC, telemetry charts, mobile responsive CSS, and vector icons.

Section 4: Iteration 3 - QA, Cloud & Final Defense (Weeks 10 - 12) — Automated test suites, zero-cost cloud deployment on Vercel/Render, and final defense.

- [Week 10: Automated Testing & Security Audits](#) — Jest/PyTest unit & integration tests, static security scans, and dependency vulnerability fixes.

- [Week 11: Zero-Cost Cloud Deployment & CI/CD](#) — Live cloud deployment on Vercel + Render + MongoDB Atlas, and GitHub Actions CI.

- [Week 12: Final Project Report & Live Defense](#) — Consolidated Final FYP Report, user manuals, code cleanup, and live presentation defense.

Section 5: Standard Weekly Project Lab Report Template — Ready-to-use markdown/PDF template with task logs, code metrics, and instructor rubric.

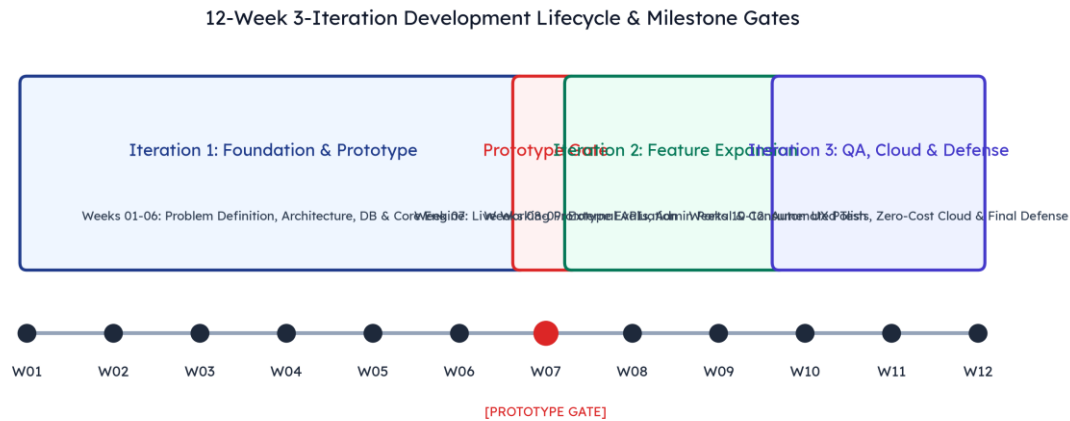
Section 6: Category-Specific Engineering Hints (10 Domains) — Actionable tips for AI, EdTech, DevTools, FinTech, Healthcare, E-Commerce, IoT & Security.

Section 7: Zero-Cost Cloud Architecture Setup Guide — Step-by-step instructions for hosting on MongoDB Atlas, Render, and Vercel for free.

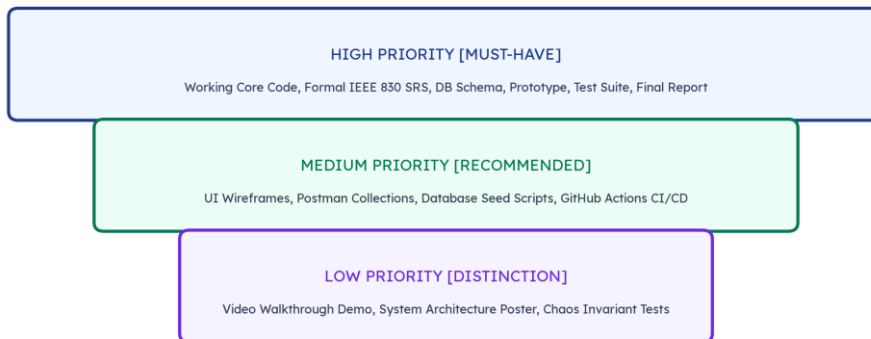
Section 8: Deliverable Grading Rubric & Quality Matrix — Clear grading criteria for Distinction, Strong, Passing, and Remediation tiers.

Section 1: The 3-Iteration Development Framework

To eliminate end-of-semester stress, the 12-week development lifecycle is divided into three distinct iterative cycles:



Deliverable Priority Hierarchy & Grade Impact



Section 2: Iteration 1 — Foundation & Working Prototype (Weeks 01 - 07)

Week 01: Problem Discovery & User Personas

[Phase: Project Inception & Requirements Gathering]

Why This Matters: Before writing a single line of code, you must clearly understand who you are building for and what specific problem your software will solve.

Action Steps for Your Team:

- Interview at least 3 target users (students, retail managers, or developers) to identify their core operational friction.
- Create 2 detailed User Personas specifying primary user roles, daily tasks, and pain points.
- Write a clear 1-paragraph Problem Statement and identify the 5 core features required for your Minimum Viable Product (MVP).

HIGH PRIORITY [MUST-HAVE]	Project Charter & Problem Statement (1-2 pages) + User Persona Profiles.
MEDIUM PRIORITY [RECOMMENDED]	Technical Feasibility Matrix (validating that zero-cost free-tier APIs and databases support the concept).
LOW PRIORITY [DISTINCTION]	Competitor Feature Comparison Matrix.

STARTER BLUEPRINT: 'Our software enables [Target User Persona] to solve [Specific Pain Point] through [Primary Feature Engine], unlike existing manual methods which cause [Key Inefficiency].'

ROOKIE TRAP: Attempting to build an all-in-one system for everyone. Focus on ONE specific target audience and ONE core problem.

Week 02: Software Requirements Specification (SRS)

[Phase: Requirements Engineering & Formal Specification]

Why This Matters: The SRS serves as the official architectural contract for your project, preventing scope creep and ensuring structural clarity.

Action Steps for Your Team:

- Decompose system functionality into formal Use Case specifications with preconditions, primary success flows, and alternative paths.
- Define quantifiable non-functional requirements (e.g., API response times < 500ms, bcrypt password hashing, 99.9% uptime).
- Construct a formal Use Case Diagram visualizing all user interactions with system boundaries.

HIGH PRIORITY [MUST-HAVE]	Formal IEEE 830 Software Requirements Specification (SRS) Document + Use Case Diagram.
MEDIUM PRIORITY [RECOMMENDED]	Requirements Traceability Matrix mapping business requirements to functional modules.
LOW PRIORITY [DISTINCTION]	Formal Data Dictionary defining entity attributes and validation constraints.

STARTER BLUEPRINT: Structure your SRS using standard IEEE 830 sections: 1. Introduction, 2. Overall Description, 3. Specific Functional & Non-Functional Requirements.

ROOKIE TRAP: Writing vague requirements such as 'The app must be fast'. Always specify measurable metrics (e.g., 'Page render latency under 1.5 seconds').

Week 03: High-Level Architecture & Cloud Topology

[Phase: High-Level Architectural Design (HLD)]

Why This Matters: Establishing clean separation between your frontend, backend API, and database layers early prevents costly refactors during implementation.

Action Steps for Your Team:

- Draw a High-Level 3-Tier Architecture Diagram defining the Client Tier (Vercel), Application Tier (Render), and Database Tier (MongoDB Atlas).
- Construct Data Flow Diagrams (DFD Level 0 Context and DFD Level 1 Module Decompositions).
- Specify inter-service communication protocols (REST over HTTPS with structured JSON payloads).

HIGH PRIORITY [MUST-HAVE]	High-Level Architecture Document with System Block Diagrams + DFD Level 0 and Level 1.
MEDIUM PRIORITY [RECOMMENDED]	Zero-Cost Cloud Deployment Specification Sheet.
LOW PRIORITY [DISTINCTION]	Architectural Decision Record (ADR) documenting backend and database selection rationale.

STARTER BLUEPRINT: Draw a 3-tier flow: [Next.js Client] --(REST / HTTPS)--> [FastAPI / Express API Router] --(Mongoose Driver)--> [MongoDB Atlas Cluster].

ROOKIE TRAP: Selecting paid cloud services (e.g., AWS EC2) that incur surprise billing. Stick strictly to zero-cost free tiers.

Week 04: Database Modeling & UI Wireframes

[Phase: Low-Level Design & Interface Architecture]

Why This Matters: Having your database schemas and user interface flows designed prior to coding speeds up development by over 300%.

Action Steps for Your Team:

- Construct the Database Entity-Relationship Diagram (ERD) or MongoDB collection schemas with indexing definitions.
- Specify the complete REST API Contract (HTTP methods, endpoints, request schemas, and response status codes).
- Design low-fidelity wireframes for the 4-6 primary application screens.

HIGH PRIORITY [MUST-HAVE]	Complete Database Schema Specification (ERD / Mongoose Models) + Formal API Endpoint Contract List.
MEDIUM PRIORITY [RECOMMENDED]	Figma Screen Flow Wireframes & Component Interaction Catalog.
LOW PRIORITY [DISTINCTION]	Postman API Collection with pre-configured

	environment variables.
--	------------------------

STARTER BLUEPRINT: Define Mongoose models with required types, unique indexes, and timestamps: ``const ModelSchema = new Schema({ ... }, { timestamps: true });``.

ROOKIE TRAP: Writing frontend views before modeling the database. Always define the data schema first so UI forms have exact fields to submit.

Week 05: Project Scaffolding & Core Auth Layer

[Phase: Implementation Cycle 1 (Environment Setup & Auth)]

Why This Matters: Initializing a clean directory structure and verified authentication layer provides the secure baseline for all project features.

Action Steps for Your Team:

- Initialize the Git repository with branch protection rules (``main``, ``dev``, ``feature/*``) and standard ``.gitignore``.
- Set up project scaffolding (Next.js/React frontend and Express/FastAPI backend).
- Establish cloud database connectivity to MongoDB Atlas with connection caching.
- Implement secure multi-tenant user authentication (signup, login, bcrypt hashing, JWT token session management).

HIGH PRIORITY [MUST-HAVE]	Verified Git Repository with Clean Scaffolding + Functional Authentication System (Signup, Login, Protected Route Guard).
MEDIUM PRIORITY [RECOMMENDED]	Database Seed Script automatically generating test user records and sample data.
LOW PRIORITY [DISTINCTION]	Docker-Compose configuration for standardized local development.

STARTER BLUEPRINT: Organize the backend into 4 layers: ``/routes`` (endpoints), ``/controllers`` (HTTP handling), ``/services`` (business logic), and ``/models`` (schemas).

ROOKIE TRAP: Hardcoding database credentials or JWT secrets in source code. Always use ``.env`` files and add them to ``.gitignore``.

Week 06: Core Business Logic & Vertical Slice

[Phase: Implementation Cycle 1 (Core Engine Development)]

Why This Matters: Connecting the user interface to the backend API and database for the primary feature proves that the foundational architecture is functional.

Action Steps for Your Team:

- Implement the primary business calculation engine (e.g., transaction round-up calculation, document parser, or core booking logic).
- Build client-side forms and connect them to backend routes via HTTP clients (``fetch`` / ``axios``).
- Implement server-side input validation schemas (Zod or Pydantic) with descriptive user error feedback.
- Verify end-to-end data persistence and real-time UI state updates upon successful submission.

HIGH PRIORITY [MUST-HAVE]	Functional Core Business Logic Engine + End-to-End
---------------------------	--

	Working Feature (UI -> API -> DB persistence).
MEDIUM PRIORITY [RECOMMENDED]	Server-side Input Validation Layer with user-friendly error banners.
LOW PRIORITY [DISTINCTION]	Optimistic UI state updates for immediate client responsiveness.

STARTER BLUEPRINT: Use async/await try-catch handlers: ``try { const res = await api.post('/resource', data); updateState(res.data); } catch (err) { handleError(err); }``.

ROOKIE TRAP: Developing frontend views in isolation using mock data. Always build end-to-end vertical slices that connect all 3 layers.

Week 07: Live Working Prototype Milestone Gate

[Phase: Mid-Term Prototype Demonstration & Smoke Testing]

Why This Matters: Demonstrating a live working prototype at mid-term confirms progress, validates architecture, and gathers supervisor feedback.

Action Steps for Your Team:

- Execute an end-to-end smoke test: Register a new student account, authenticate, create a domain record, modify it, and query it.
- Eliminate broken navigation links, unhandled edge-case errors, and layout visual bugs.
- Prepare a concise 5-minute live demonstration walkthrough and slide deck for supervisor review.

HIGH PRIORITY [MUST-HAVE]	Live Functional Working Prototype Demonstration + Mid-Term Progress Report & Slide Deck.
MEDIUM PRIORITY [RECOMMENDED]	Smoke Test Execution Checklist and Verification Log.
LOW PRIORITY [DISTINCTION]	Recorded 3-Minute Video Walkthrough of the working prototype.

STARTER BLUEPRINT: Structure the demo into 3 phases: 1. Problem statement (1 min), 2. Live primary user workflow walkthrough (3 mins), 3. Next sprint roadmap (1 min).

ROOKIE TRAP: Presenting static slides instead of executing running code. Supervisors evaluate live, functional software.

Section 3: Iteration 2 — Feature Expansion & Governance (Weeks 08 - 09)

Week 08: Secondary Modules & Third-Party APIs

[Phase: Implementation Cycle 2 (External Services & Secondary Features)]

Why This Matters: With the core engine proven, integrating external APIs (AI, messaging, maps, payments) elevates your application into a production-grade tool.

Action Steps for Your Team:

- Integrate external services (e.g., Gemini 1.5 Flash API, Meta WhatsApp Cloud API, Mapbox, or Stripe).
- Implement defensive error-handling wrappers and fallback states for third-party API timeouts.
- Build secondary user features including full-text search, multi-field filtering, and PDF/CSV data exports.

HIGH PRIORITY [MUST-HAVE]	Integrated Third-Party API Adapters + Functional Secondary Feature Modules.
MEDIUM PRIORITY [RECOMMENDED]	Asynchronous Background Task or Scheduled Job Engine.
LOW PRIORITY [DISTINCTION]	Cryptographically Verified Webhook Ingestion Pipeline.

STARTER BLUEPRINT: Enforce timeout guards on all external API requests: ``const response = await fetch(url, { signal: AbortSignal.timeout(8000) });``.

ROOKIE TRAP: Storing third-party API keys in client-side code where they are exposed. Always invoke external APIs through your backend.

Week 09: Admin Workstation & Consumer UX Polish

[Phase: Implementation Cycle 2 (Governance & User Experience)]

Why This Matters: A dedicated Admin portal provides essential governance capabilities, while consumer UX polish ensures high visual appeal and usability.

Action Steps for Your Team:

- Construct a protected Admin Portal with Role-Based Access Control (RBAC) to manage users and audit system activity.
- Implement interactive analytics dashboards rendering key business metrics using SVG charts.
- Refine UI responsiveness across mobile, tablet, and desktop viewports, enforcing clean vector SVG icons.

HIGH PRIORITY [MUST-HAVE]	Functional Admin Dashboard with Role-Based Access Control + Responsive UI Polish across Viewports.
MEDIUM PRIORITY [RECOMMENDED]	Aggregated System Telemetry & Activity Log Views.
LOW PRIORITY [DISTINCTION]	Theme Switching Engine supporting Light and Dark modes.

STARTER BLUEPRINT: In backend auth middleware: ``if (req.user.role !== 'admin') return res.status(403).json({ error: 'Admin access required' });``.

ROOKIE TRAP: Rendering textual pseudo-icons like ``[x]`` or ``[ ]`` in UI controls. Exclusively render clean SVG icons.

Section 4: Iteration 3 — QA, Cloud Deployment & Final Defense (Weeks 10 - 12)

Week 10: Automated Testing & Security Audits

[Phase: Verification, QA & Security Auditing]

Why This Matters: Automated test suites act as a safety net, guaranteeing that new changes do not break existing features or compromise user security.

Action Steps for Your Team:

- Write automated unit tests for all domain calculation algorithms and utilities using Jest or PyTest.
- Implement API integration tests for primary backend routes using in-memory mock databases.
- Conduct a comprehensive security audit checking for SQL/NoSQL injection, CORS misconfigurations, and vulnerable packages (``npm audit``).

HIGH PRIORITY [MUST-HAVE]	Automated Test Suite with $\geq 70\%$ code coverage on core domain logic + Security Vulnerability Remediation Report.
MEDIUM PRIORITY [RECOMMENDED]	End-to-End User Flow Test Scripts (Playwright or Cypress).
LOW PRIORITY [DISTINCTION]	Chaos Invariant & Network Flapping Test Suite.

STARTER BLUEPRINT: Author Jest unit tests: ``test('calculates round up correctly', () => { expect(calculateRoundUp(3.40)).toBe(0.60); });``.

ROOKIE TRAP: Writing tests only for happy paths. Always test invalid inputs, empty payloads, and boundary conditions.

Week 11: Zero-Cost Cloud Deployment & CI/CD

[Phase: Deployment & Cloud Performance Optimization]

Why This Matters: Deploying the complete application to live public URLs ensures it is accessible on any device during evaluations and presentations.

Action Steps for Your Team:

- Deploy MongoDB database to MongoDB Atlas Free Tier with production access credentials.
- Deploy backend API service to Render Free Tier and configure health-check ping keepalives.
- Deploy frontend web application to Vercel Free Tier with automated HTTPS and custom subdomain.
- Configure GitHub Actions CI workflow to run linters and automated tests on every pull request.

HIGH PRIORITY [MUST-HAVE]	Live Public Cloud Deployment URLs (Frontend & Backend API) + Functional GitHub Actions CI Pipeline.
MEDIUM PRIORITY [RECOMMENDED]	Performance & API Latency Benchmark Report.
LOW PRIORITY [DISTINCTION]	Automated Keepalive Ping Service preventing free-tier cold starts.

STARTER BLUEPRINT: Create ``.github/workflows/ci.yml`` running ``npm install``, ``npm run lint``, and ``npm test`` on all push events.

ROOKIE TRAP: Postponing cloud deployment until the day before defense. Deploy in Week 11 to identify and fix cloud issues early.

Week 12: Final Project Report & Live Defense

[Phase: Project Closure & Final Evaluation Defense]

Why This Matters: Consolidating all engineering documentation and mastering the live demonstration ensures a confident, successful final evaluation.

Action Steps for Your Team:

- Consolidate all 12 weekly lab reports into the comprehensive Final FYP Project Report.
- Author the Developer Setup Guide and User Operational Manual.
- Sanitize the codebase: remove temporary debug logs, obsolete files, and unused dependencies.
- Conduct multiple practice defense dry runs to master demonstration timing and question responses.

HIGH PRIORITY [MUST-HAVE]	Final Comprehensive FYP Project Report + Polished Production Codebase + Presentation Slide Deck & Live Demo.
MEDIUM PRIORITY [RECOMMENDED]	Comprehensive User Manual & API Reference Documentation.
LOW PRIORITY [DISTINCTION]	High-Resolution System Architecture Poster (PDF).

STARTER BLUEPRINT: Structure final presentation: 1. Problem & Persona (1 min), 2. Live feature demo (3 mins), 3. Architecture & DB (1 min), 4. Q&A (2 mins).

ROOKIE TRAP: Forgetting to verify live cloud deployment on external mobile data. Always test outside local development networks.

Section 5: Standard Weekly Project Lab Report Template

Students must submit a standalone lab report each week adhering to this standard structure:

WEEKLY PROJECT LAB REPORT SCHEMA

- 1. Project Metadata: Project ID (#001 - #100), Project Title, Student Names, Roll Numbers, GitHub Repository URL, and Active Branch.
- 2. Implementation Summary: Current SDLC phase, planned objectives vs. actual achievements, and roadmap status ([ON TRACK / AHEAD / DELAYED]).
- 3. Engineering Task Matrix: Table of completed tasks with Task ID, descriptions, Git commit hashes, assigned members, and verification status ([PASS]).
- 4. Architectural Highlight: 1 structural design decision + 1 clean code snippet (10-15 lines).
- 5. Technical Obstacles & Resolution: 1 challenge encountered, root cause analysis, and applied engineering patch.
- 6. Deliverables Check: Verified checkboxes for High, Medium, and Low priority deliverables.
- 7. Next Sprint Goals: 3 specific technical objectives for the following week.
- 8. Academic Supervisor Evaluation: Weekly score (/100), feedback notes, and signature line.

Section 6: Category-Specific Hints & Guardrails (10 Domains)

1. AI Copilots & Autonomous Agents (#001 - #010)

- **Token Optimization:** Avoid submitting large unparsed documents; chunk text before invoking LLM endpoints.
- **Deterministic JSON:** Require the AI model to respond in structured JSON matching predefined schemas.
- **Resilient Fallbacks:** Implement retry handlers with exponential backoff for external API timeouts.

2. EdTech & Student Productivity (#011 - #020)

- **Authentic Student Records:** Persist verified student attributes (email, name, student ID, major, year).
- **Context Chunking:** Segment documents into 400-word blocks with 50-word overlaps for vector indexing.
- **Role Separation:** Isolate student and instructor capabilities using role-based route middleware.

3. Developer Tools & DevOps Automation (#021 - #030)

- **CLI Standards:** Return standard POSIX exit codes (0 for success, non-zero for error) and clean ASCII tables.
- **Sandboxed Parsing:** Evaluate untrusted user code and AST analyzers within isolated worker threads.
- **Webhook Idempotency:** Track event IDs in persistence layers to prevent duplicate processing on retries.

4. Local Commerce & SMB Operations (#031 - #040)

- **Offline Resilience:** Utilize local storage queues so orders persist during network disconnections.
- **Real-Time Feeds:** Implement WebSockets or Server-Sent Events for order preparation and kitchen displays.
- **Touch Ergonomics:** Design point-of-sale action buttons with large 48px touch targets.

5. FinTech & Micro-Finance (#041 - #050)

- **Atomic Transactions:** Wrap all balance updates within atomic transactions (``withTransaction()``).
- **Integer Precision:** Store monetary figures in smallest currency units (cents/paisa as integers) to prevent float errors.
- **Append-Only Ledgers:** Maintain an immutable transaction history ledger for audit verification.

6. Healthcare & Telemedicine (#051 - #060)

- **Data Sanitization:** Redact patient names and contact details prior to external AI processing.
- **Image Preprocessing:** Apply OpenCV grayscale thresholding and noise reduction before OCR extraction.
- **Verified Disclaimers:** Display clear notices stating the software provides decision support, not medical diagnosis.

7. E-Commerce & Creator Economy (#061 - #070)

- **Atomic Inventory Locking:** Enforce atomic decrement conditions (``quantity >= orderedQty``) during checkout.
- **Asset Optimization:** Compress and serve product images in WebP format with responsive srcset scaling.
- **Payment Verification:** Verify payment gateway cryptographic signatures prior to order fulfillment.

8. Smart Campus & Mobility IoT (#071 - #080)

- **Geospatial Indexing:** Apply MongoDB ``2dsphere`` indexes for rapid geospatial distance queries.
- **Battery Optimization:** Throttle mobile GPS polling rates when stationary via geofencing.

- **Resilient Connections:** Implement automatic reconnect loops for IoT telemetry and vehicle sockets.

9. Cybersecurity & LegalTech (#081 - #090)

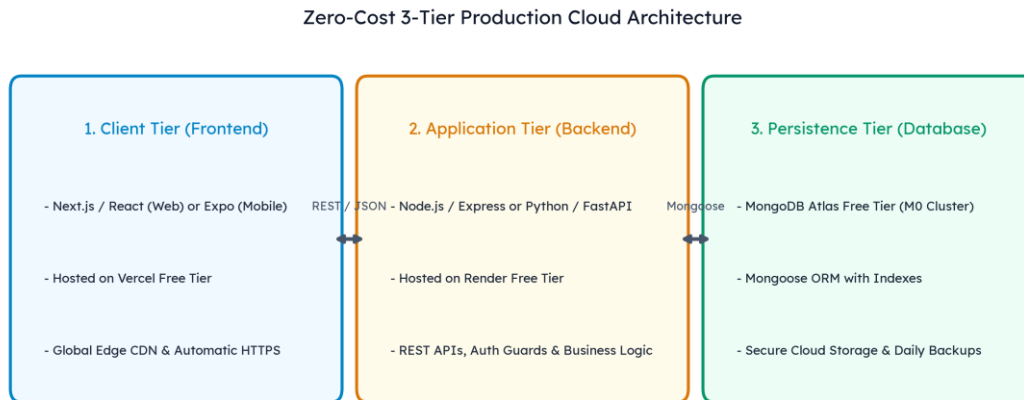
- **Client-Side Cryptography:** Encrypt sensitive documents client-side using WebCrypto AES-GCM-256.
- **ReDoS Mitigation:** Sanitize regular expressions against catastrophic backtracking on large documents.
- **Breach Detection:** Utilize k-Anonymity (submitting only SHA-1 5-character prefixes) for password checks.

10. Productivity & Remote Work (#091 - #100)

- **Optimistic UI Updates:** Update task checkboxes immediately on client UI, reverting only on server error.
- **Keyboard Navigation:** Implement keyboard shortcuts (`Ctrl+Enter`, `Esc`) for high-frequency workflows.
- **Conflict Resolution:** Enforce Last-Write-Wins timestamps to manage concurrent collaborative updates.

Section 7: Zero-Cost Cloud Architecture Setup Guide

Every project can be deployed to production cloud infrastructure without incurring any hosting fees:



Step 1: Database Setup on MongoDB Atlas: Provision a free MongoDB Atlas M0 cluster (512MB storage, automated backups, encryption). Copy connection URI to ``.env``.

Step 2: Backend API Deployment on Render: Link GitHub repository to Render as a Free Web Service. Configure environment variables and expose a ``.health`` endpoint.

Step 3: Frontend Web Deployment on Vercel: Import repository into Vercel. Builds, SSL certificates, and global edge CDN are provisioned automatically for zero cost.

Step 4: Continuous Testing on GitHub Actions: Add a ``.github/workflows/ci.yml`` pipeline to automatically execute test suites on every pull request.

Section 8: Deliverable Grading Rubric & Quality Matrix

Grade Tier	Required Deliverables	Evaluation Criteria
Distinction Grade (85% - 100%)	All HIGH + All MEDIUM + Selected LOW deliverables completed across all 12 weeks.	Live public cloud deployment URL, >= 70% automated test coverage, complete IEEE 830 SRS, automated CI/CD pipeline, and weekly lab reports with code metrics.
Strong Grade (70% - 84%)	All HIGH + Majority of MEDIUM deliverables completed.	Working local or staging prototype, clean database schema with indexes, documented API routes, basic unit tests, and complete SRS.
Passing Grade (50% - 69%)	All HIGH mandatory deliverables completed.	Basic working application fulfilling core user stories, functional login, and database persistence.
Failing Grade (< 50%)	Incomplete HIGH deliverables or broken core engine.	Application fails to compile, database does not persist data, or unsubmitted weekly lab reports.